Throughout the Contact Service, Task Service, and Appointment Service projects, I used the project requirements as a checklist to guide my testing. My goal was to make sure every major requirement had a corresponding JUnit test. As I progressed through the course, I built on what I learned from previous projects. When I started the Task Service project, I used my Contact Service project as a guide because the structure and requirements were similar. Later, when I completed the Appointment Service project, I was more comfortable creating tests and understanding how to verify that the code met the requirements. These projects helped me gain a better understanding of software testing and showed me how important testing is for creating reliable software.

My testing approach was closely aligned with the software requirements because each test was created to verify a specific requirement. For the Contact Service project, I tested creating contacts, updating contact information, and deleting contacts. I also tested invalid data to make sure the program would reject information that did not meet the requirements. For the Task Service project, I tested creating tasks, updating task names and descriptions, deleting tasks, and preventing duplicate task IDs. For the Appointment Service project, I tested adding appointments, deleting appointments, preventing duplicate appointment IDs, and making sure appointments could not be created with dates in the past. Instead of only testing situations where everything worked correctly, I also created tests that intentionally used invalid information. This allowed me to confirm that the validation rules were working properly and that the program would not accept incorrect data.

I believe my JUnit tests were effective because they covered the major requirements for each project and produced strong coverage results. The Contact Service project achieved 90.7% code

coverage, the Appointment Service project achieved 81.7% code coverage, and the Task Service project achieved over 80% coverage. These coverage percentages showed that most of the code was being executed while the tests were running. In addition to the coverage reports, all of my tests passed successfully. Seeing both strong coverage results and successful test executions reassured me that the programs were meeting the project requirements. While no test suite is perfect, the combination of code coverage and requirement-based testing showed that the most important parts of the applications were being tested.

I made sure my code was technically sound by creating tests that checked both valid and invalid situations. For example, in the Task Service project, I created tests that verified invalid task IDs, task names, and task descriptions would generate exceptions when they violated the requirements. I also used assertions to verify that updates were occurring correctly. One example was using assertEquals("State Park", task.getName()) after updating a task name to make sure the change was successful. In the Appointment Service project, I used assertThrows() to verify that duplicate appointment IDs and past appointment dates would be rejected. I also used assertDoesNotThrow() when testing valid appointments to verify that they could be added successfully. These tests confirmed that the code was following the requirements and handling different situations correctly.

I also tried to keep my tests organized and easy to understand. I created separate tests for different features instead of putting everything into one test method. For example, I had individual tests for updating task information, deleting tasks, and checking for duplicate IDs. This made it easier to identify what was causing a problem if a test failed and helped keep the code easier to follow. Throughout the projects, Eclipse also helped identify errors and provided feedback when something was not working correctly, which made troubleshooting much easier.

The primary testing technique I used throughout these projects was unit testing. Unit testing focuses on testing individual parts of a program separately to ensure they work correctly on their own. Since the Contact Service, Task Service, and Appointment Service projects were built around individual classes and services, unit testing was the most appropriate technique. By using JUnit, I was able to verify that constructors, validation rules, update methods, and service methods behaved as expected. Unit testing helped me catch mistakes early before they became larger problems.

Another testing technique I used was dynamic testing. Dynamic testing occurs when the software is executed and observed while it is running. Every time I ran my JUnit tests, I was performing dynamic testing because I was actively executing the code and observing the results. This allowed me to verify that the services worked correctly during execution rather than simply assuming they would work by reading the code. Running the code allowed me to catch issues that I may not have noticed by simply reading through it.

One testing technique that I did not use was integration testing. Integration testing focuses on verifying that multiple components of a system work together correctly. The projects in this course were primarily focused on testing individual services rather than testing interactions between multiple systems, so unit testing was a better fit for the assignments. While integration testing was not necessary for these projects, I understand its value in larger systems. It can help identify problems that may not appear when testing individual classes, especially when different parts of a program need to exchange information and work together to complete a task.

Through these projects, I learned that testing requires a different mindset than writing code. When developing software, it is easy to focus on making the code work under normal conditions. Testing requires thinking about what could go wrong and identifying situations where users might enter incorrect information. Throughout the projects, I intentionally tested invalid IDs, invalid field lengths, duplicate IDs, and past dates because I wanted to make sure the programs could handle those situations correctly. Instead of assuming everything would work correctly, I spent more time looking for situations that could cause problems.

These projects also helped me realize how connected different parts of a program can be. Even though the Contact Service, Task Service, and Appointment Service projects were not very large, I noticed that a small change in one area could affect other parts of the code. For example, if the validation rules in a constructor were not working correctly, it could cause problems later when the service methods tried to create or manage objects. Working through these projects made me pay more attention to how different pieces of code rely on one another instead of only focusing on whether a single method worked by itself.

To limit bias while reviewing my code, I tried to approach testing from the perspective of a user rather than the developer who wrote the program. When developers test their own code, it is easy to assume the code works because they understand how it was intended to function. To avoid this, I deliberately created tests with invalid data and edge cases instead of only testing successful situations. For example, I tested duplicate IDs, invalid field lengths, and past appointment dates to make sure the programs handled those situations correctly. Doing this helped me look at my projects more critically and made me more confident that the final versions were working as intended. If I had only tested successful scenarios, I could have overlooked problems that would affect users later.

Overall, these projects gave me a much better understanding of software testing than I had at the beginning of the course. When I started the Contact Service project, I was mostly focused on getting the tests to run correctly. By the time I completed the Appointment Service project, I had a better understanding of why testing is important and how it helps verify requirements. Going forward, I plan to continue using the requirements as a checklist, testing both valid and invalid inputs, and looking for potential issues before submitting my work. These projects showed me that spending extra time on testing upfront is worth it because it can prevent much bigger problems later.